Development Setup Guide

Vision-Based Illegal Waste Detection Using Geotagged Smartphone Images and YOLOv11

This guide sets up everything you need across your two devices:

- **iPhone 15 Pro Max** — your data collection device (takes the geotagged photos)
- **Windows laptop with an NVIDIA GTX 1050 (4GB VRAM)** — your development machine (code, local model training/testing, GIS mapping, backend)

It also plans ahead for the **citizen reporting website** you want to build alongside the core detection system: a public site where anyone can photograph a suspected waste pile, have it analysed automatically, and — if confirmed — trigger an alert to a municipality with the location and time. Part 8 covers that in detail.

Follow the steps in order. Each step says exactly what to install and how.

Part 1 — iPhone 15 Pro Max: capturing geotagged photos

You don't need any special app for this. The built-in Camera app already embeds GPS coordinates and a timestamp into every photo's EXIF metadata, as long as Location Services is enabled for the Camera app.

1.1 Turn on location tagging

1. Open **Settings** → **Privacy & Security** → **Location Services**.
2. Make sure **Location Services** is toggled ON at the top.
3. Scroll down to **Camera**, tap it, and select **While Using the App**.

1.2 Turn on high-accuracy location (optional but recommended)

1. Settings → Privacy & Security → Location Services → System Services.
2. Make sure **Precise Location** is enabled for the Camera (this is usually on by default on iOS 17+).

1.3 Take your photos

- Use the native Camera app in standard **Photo** mode (not Live Photo, not Portrait — plain photos keep EXIF simplest to parse).
- Walk around your target sites in Harare/Bulawayo: illegal dumps, roadsides, clean areas, bins, rivers, markets, residential areas — matching your dataset classes.
- Try to vary lighting, distance, and angle for each waste type — this directly improves model generalisation.

1.4 Get the photos onto your Windows laptop

You have three good options; pick whichever is easiest for you:

- **Option A — USB cable (most reliable, keeps EXIF intact):**
  1. Connect the iPhone to the laptop with a USB-C to Lightning (or USB-C to USB-C, if you have the newer cable) cable.
  2. On the iPhone, tap **Trust This Computer** when prompted.
  3. On Windows, open **File Explorer** → **This PC** → your iPhone should appear as a device.
  4. Navigate to `Internal Storage\DCIM` and copy the photos to a folder on your laptop, e.g. `C:\WasteProject\raw_images`.
- **Option B — iCloud Photos + iCloud for Windows:**
  1. Install **iCloud for Windows** from the Microsoft Store.
  2. Sign in with your Apple ID and enable **iCloud Photos**.
  3. Your photos sync automatically to `C:\Users\<you>\iCloudPhotos`.
  4. ⚠️ Note: iCloud Photos can sometimes strip or alter EXIF data depending on settings — after transferring a test batch, check a sample image's metadata (Part 2.6 below) before doing a full transfer.
- **Option C — Google Photos app (free, keeps EXIF):**
  1. Install the **Google Photos** app from the App Store on your iPhone.
  2. Turn on backup with **Original quality**.
  3. On your laptop, go to photos.google.com and download the images.

Recommendation: Use Option A (USB) for your main dataset — it's the most reliable way to guarantee GPS/timestamp metadata survives the transfer.

Part 2 — Windows laptop: core development environment

2.1 Your hardware: NVIDIA GTX 1050 (4GB VRAM)

You have a dedicated GPU, so you can train locally instead of relying only on Google Colab. This is good — it means faster iteration during development, since you won't need to re-upload datasets to the cloud every time. A few things to know given your specific card:

- The GTX 1050 is CUDA-capable (compute capability 6.1), so it works fine with current PyTorch/CUDA releases.
- **4GB of VRAM is tight** for deep learning. In practice this means:
  - Use the **nano or small** YOLOv11 variants (`yolo11n.pt` or `yolo11s.pt`) — the larger variants (`m`, `l`, `x`) will likely run out of memory on 4GB.
  - Keep **image size at 640** (the default) rather than increasing it.

    - Use a **small batch size** (start with `batch=8`, drop to `batch=4` if you get an "out of memory" error).
    - Close other GPU-hungry applications (browser with many tabs, games, etc.) while training.
- **Google Colab remains useful as a backup/scale-up option** — if you later want to train larger model variants, run longer experiments, or free up your laptop while it trains, Colab's free T4 GPU (16GB) can handle what your GTX 1050 can't. Both Part 2 (local) and Part 4 (Colab) are kept in this guide for that reason.

2.1.1 Install/update your NVIDIA driver

1. Download the latest driver for your GTX 1050 from **nvidia.com/drivers** (select GeForce → GTX 1050 → Windows). Alternatively, install **NVIDIA GeForce Experience** which keeps drivers updated automatically.
2. Run the installer with default options and restart your laptop if prompted.
3. Verify it worked — open **Command Prompt** and run:

```
nvidia-smi
```

This should print a table showing your GPU name, driver version, and a **CUDA Version** in the top right (e.g. "CUDA Version: 12.4"). Note this number down — it tells you the *maximum* CUDA version your driver supports (you'll match PyTorch to it in step 2.5).

2.2 Install Python

1. Go to **python.org/downloads** and download **Python 3.11** (3.11 has the best current compatibility with PyTorch/Ultralytics; avoid 3.13 for now).
2. Run the installer.
3. ⚠️ **Critical step:** On the first installer screen, tick the checkbox **"Add python.exe to PATH"** at the bottom before clicking Install.
4. Click **Install Now**.
5. Verify it worked — open **Command Prompt** (search "cmd" in the Start menu) and run:

```
python --version
pip --version
```

You should see version numbers, not an error.

2.3 Install Git for Windows

1. Download from **git-scm.com/download/win**.
2. Run the installer — the default options are fine for all screens; just keep clicking **Next** then **Install**.
3. Verify:

```
git --version
```

2.4 Install Visual Studio Code

1. Download from **code.visualstudio.com**.
2. Run the installer. On the "Select Additional Tasks" screen, tick **"Add to PATH"** and **"Register Code as an editor for supported file types"**.
3. Open VS Code, go to the **Extensions** panel (left sidebar, square icon), and install:
   - **Python** (by Microsoft)
   - **Pylance** (by Microsoft)
   - **Jupyter** (by Microsoft) — lets you run notebook-style cells, useful for experimenting with the model

2.5 Create your project folder and a virtual environment

A virtual environment keeps this project's Python packages separate from everything else on your system — always do this.

1. Open **Command Prompt** and create your project structure:

```
mkdir C:\WasteProject
cd C:\WasteProject
mkdir raw_images dataset scripts models notebooks app
```

2. Create the virtual environment:

```
python -m venv venv
```

3. Activate it (you'll need to do this every time you open a new terminal to work on the project):

```
venv\Scripts\activate
```

Your prompt should now show `(venv)` at the start of the line. 4. Open the folder in VS Code:

```
code .
```

In VS Code, press `Ctrl+Shift+P` → type "Python: Select Interpreter" → choose the one inside `.\venv\`.

2.6 Install PyTorch with GPU (CUDA) support

This step matters for you specifically because you want to use your GTX 1050 for training, so install the CUDA build of PyTorch **before** installing Ultralytics.

1. Check the CUDA version your driver reported from `nvidia-smi` in step 2.1.1. Most current GTX 1050 drivers support **CUDA 12.1** or newer, so we'll use that build.
2. With `(venv)` active, run:

```
   pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
```

3. Verify GPU support is working:

```
   python -c "import torch; print('CUDA available:',
torch.cuda.is_available()); print('GPU:', torch.cuda.get_device_name(0) if
torch.cuda.is_available() else 'none')"
```

You should see:

```
   CUDA available: True
   GPU: NVIDIA GeForce GTX 1050
```

If it prints `False`, don't worry — it just means Ultralytics/inference will fall back to CPU (slower but still functional). Double-check your driver installed correctly (step 2.1.1) and that you used the `cu121` index URL above before troubleshooting further.

2.7 Install the remaining core Python packages

Still with `(venv)` active, run:

```
pip install ultralytics opencv-python numpy pandas matplotlib pillow exifread
```

What each one does:

- **ultralytics** — the official YOLOv11 package (training, inference, export). Since PyTorch is already installed with CUDA support, Ultralytics will automatically detect and use your GPU.
- **opencv-python** — image loading/processing
- **numpy** / **pandas** — data handling
- **matplotlib** — plotting training curves and results
- **pillow** — image manipulation (used alongside OpenCV)
- **exifread** — reads GPS/timestamp metadata out of your iPhone photos

Quick test that everything works and is using your GPU:

```
python -c "from ultralytics import YOLO; m = YOLO('yolo11n.pt'); r =
m.train(data='coco8.yaml', epochs=1, device=0); print('YOLOv11 trained one
epoch on GPU successfully')"
```

This downloads a tiny sample dataset (`coco8`) and runs one training epoch on your GPU (`device=0` forces GPU use) purely to confirm the whole pipeline — driver, CUDA, PyTorch, Ultralytics — works together end to end. If `device=0` throws an error, drop it (Ultralytics will use CPU) and revisit step 2.6.

When you get to real training on your own dataset later, given your 4GB VRAM, start with:

```python
model = YOLO("yolo11n.pt")
model.train(data="data.yaml", epochs=100, imgsz=640, batch=8, device=0)
```

and reduce `batch` to 4 if you see a `CUDA out of memory` error.

2.8 Check that an iPhone photo's GPS data is readable

```
python -c "import exifread;
f=open(r'C:\WasteProject\raw_images\test.jpg','rb');
tags=exifread.process_file(f); print(tags.get('GPS GPSLatitude'),
tags.get('GPS GPSLongitude'))"
```

(Replace `test.jpg` with an actual filename from your transferred photos.) If this prints coordinate values instead of `None None`, your geotagging pipeline is working end to end.

Part 3 — Annotation tool: Roboflow

Roboflow is the easiest annotation tool to start with — it's entirely browser-based, has a generous free tier, and exports directly in YOLO format, so no separate install is needed.

1. Go to **roboflow.com** and sign up for a free account.
2. Create a new project → choose **Object Detection**.
3. Upload your transferred iPhone photos.
4. Draw bounding boxes around waste items and label them with your classes (Plastic, Organic Waste, Metal, Tyres, Construction Waste, Mixed Waste).
5. Once you've annotated enough images, use Roboflow's **Generate** → **Export Dataset** → format **YOLOv11** (or "YOLOv8 PyTorch TXT", which is compatible) to get a ready-to-train dataset folder.

Part 4 — Google Colab: free GPU training (scale-up option)

Since your GTX 1050 handles local training for smaller/nano YOLOv11 variants, you don't strictly need Colab to get started. Keep it in your toolkit for when you want to train a larger model variant, run a longer experiment without tying up your laptop, or free up your GPU for something else — it also gives you a free 16GB-VRAM GPU (T4) as a backstop.

1. Go to **colab.research.google.com** and sign in with a Google account (create one if you don't have one).
2. Create a new notebook.
3. Go to **Runtime → Change runtime type** → set **Hardware accelerator** to **GPU** (T4 is fine on the free tier).
4. In the first cell, install Ultralytics:

```python
!pip install ultralytics
```

5. Upload your Roboflow-exported dataset (either drag-and-drop into Colab's file panel, or mount Google Drive) and start training with a command like:

```python
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
model.train(data="data.yaml", epochs=100, imgsz=640)
```

6. Download the resulting `best.pt` weights file back to your laptop's `models` folder when training finishes.

Part 5 — GIS and mapping tools

QGIS (desktop GIS software):

1. Download from **qgis.org/download** → choose the Windows installer.
2. Run it with default options.

Folium (Python mapping library): With `(venv)` active:

```
pip install folium
```

No further setup needed — you'll use this to plot detection coordinates on an interactive map from Python.

Part 6 — Backend and database

With `(venv)` active, install the web framework and database driver:

```
pip install flask flask-sqlalchemy python-dotenv
```

- **Flask** — lightweight web framework for the reporting backend
- **flask-sqlalchemy** — makes it easy to work with a database from Flask
- **python-dotenv** — loads secret values (email passwords, API keys) from a local `.env` file instead of hardcoding them in your scripts

Database: SQLite needs no installation — it's built into Python's standard library (`sqlite3`) and Flask-SQLAlchemy will create the database file automatically the first time you run the app. You only need to install PostgreSQL separately later if you outgrow SQLite (unlikely for this project's scope).

This backend will end up doing double duty: it's the same Flask app that will (a) serve your citizen-facing reporting website described in Part 8, and (b) power your GIS dashboard for municipal staff. Keep that in mind when you structure `app/` — it's one project, not two.

Part 7 — Version control with Git and GitHub

1. Go to **github.com** and create a free account if you don't have one.
2. Create a new repository, e.g. `waste-detection-zimbabwe`.
3. Back in Command Prompt, inside `C:\WasteProject`:

```
   git init
   git add .
   git commit -m "Initial project structure"
   git branch -M main
   git remote add origin https://github.com/<your-username>/waste-detection-
zimbabwe.git
   git push -u origin main
```

4. Create a `.gitignore` file in `C:\WasteProject` to keep large files and the virtual environment out of GitHub:

```
venv/
raw_images/
dataset/
*.pt
__pycache__/
```

Part 8 — Citizen reporting website (planned feature)

This is the feature you described: any member of the public opens a website, takes or uploads a photo of a suspected waste pile, the system analyses it, and — if it's confirmed as illegal waste — automatically alerts a municipality or organisation with the location and time. You don't need to build this yet, but it's worth installing the extra pieces now so they're ready when you get to it, and worth understanding the design up front since it affects how you structure the detection code from the start.

8.1 How the pipeline will work, end to end

```
Citizen's browser/phone
      |  (1) opens website, taps "Report Waste"
      ▼
Camera capture / file upload (HTML5)
      |  (2) photo + browser GPS coordinates + timestamp sent to server
      ▼
Flask backend receives the image
      |  (3) runs YOLOv11 inference on the image
      ▼
   Is it waste?  ──No──▶ Reject / notify user "no waste detected"
      | Yes
      ▼
(4) Record: image, GPS lat/long, timestamp, waste type(s), confidence →
database
      ▼
(5) Trigger alert (email/SMS) to the relevant municipality/organisation
      ▼
(6) Detection also appears as a pin on the GIS dashboard (Part 5)
```

Important design point on geolocation: photos taken directly through a website (via the browser's camera capture) usually do **not** carry EXIF GPS data the way your iPhone's native Camera app does, because browsers typically strip metadata for privacy, or the image comes from a live camera stream rather than a saved photo file. So for the website specifically, don't rely on EXIF — instead capture location using the **browser's Geolocation API** (`navigator.geolocation.getCurrentPosition()`), which asks the visitor for permission and returns coordinates directly, and pair that with a **server-side timestamp** taken the

moment the image is received. This is more reliable for random citizen reports than metadata parsing, and it's a different code path from the EXIF-based approach you're using for your own iPhone-collected training dataset (Part 2.8) — you'll end up with two small, separate pieces of code: one for reading EXIF from your training images, one for reading browser geolocation on the live site.

8.2 Additional installs for this feature

With `(venv)` active:

```
pip install flask-mail
```

- **flask-mail** — sends email alerts (to a municipal inbox, for example) directly from Flask using a standard email account or a transactional email service.

For email alerts (simplest option to start with):

- You can use a free Gmail account with an **App Password** (Google Account → Security → 2-Step Verification → App Passwords) rather than your real password, and configure `flask-mail` to send through Gmail's SMTP server. This costs nothing and is enough for early testing and even a real pilot with one or two municipal contacts.

For SMS alerts (optional, add later if needed):

```
pip install twilio
```

- **Twilio** is a paid service (small per-message cost) that sends SMS directly to a phone number — useful if your target municipal contact is more likely to check a text message than an inbox. Not required for the initial version; email is a fine starting point and keeps costs at zero.

For serving the website (once ready to run it beyond just testing locally):

```
pip install waitress
```

- **Waitress** is a production-ready web server for Flask on Windows (Flask's own built-in server is fine for development but isn't meant for real traffic). You'd run the app with Waitress instead of `flask run` once you're ready to let real users hit it.

8.3 A note on hosting

Everything above runs perfectly on your laptop for development and testing. When you're ready for the public to actually access the website (rather than just you testing on `localhost`), you'll need to deploy it somewhere reachable on the internet — free/low-cost options like **Render**, **Railway**, or a university server are all reasonable choices. That's a

later-stage decision, not something to set up now — flagging it so it's on your radar as the project matures.

Suggested project folder structure

```
C:\WasteProject
├── venv/                  # virtual environment (not committed to Git)
├── raw_images/            # unannotated iPhone photos (your own dataset
collection)
├── dataset/               # Roboflow-exported annotated dataset
├── models/                # trained YOLOv11 weight files (best.pt)
├── notebooks/             # experimentation notebooks
├── scripts/               # standalone scripts (exif_extract.py, detect.py,
train.py)
├── app/                   # Flask backend — serves BOTH the GIS dashboard
and
|   ├── static/            # the citizen reporting website (Part 8)
|   ├── templates/         # HTML pages (upload form, map view)
|   ├── models.py          # database models (reports, detections)
|   ├── inference.py       # shared YOLOv11 inference logic used by both
|   |                      #   your dataset scripts AND the live website
|   ├── alerts.py          # email/SMS alert logic (Part 8.2)
|   └── main.py            # Flask app entry point
├── .env                   # secret values: email password, API keys (never
commit this)
├── .gitignore
└── README.md
```

Note the `inference.py` module deliberately sits in one place and gets reused by both your training/testing scripts and the live website — you want one detection pipeline, not two divergent copies of "is this waste or not" logic.

What's next

Once everything above is installed and the tests in Part 2.6/2.7 pass, a sensible order to tackle things is:

1. **Collect and annotate your first batch of images** (aim for at least a few hundred to start training meaningfully) using your iPhone and Roboflow.
2. **Run a first baseline training pass locally** on your GTX 1050 with `yolo11n.pt`, just to confirm the whole pipeline works end to end — dataset → training → a working `best.pt`.

3. **Build a simple inference script** (`scripts/detect.py`) that loads `best.pt`, runs it on a test image, and prints whether waste was detected plus the confidence score. This becomes the core logic that `app/inference.py` will later reuse for the website.
4. **Iterate on the model**: add more images, refine classes, retrain — scaling up to Colab if you want to try larger model variants.
5. **Only once detection is working reliably**, move on to the website (Part 8): the upload form, browser geolocation capture, and the alert-sending logic.

Let me know once you've got Python, Git, VS Code, and the CUDA-enabled PyTorch installed, and I can help you write the first script — extracting GPS/timestamp data from your iPhone photos, or the initial baseline training script, whichever you'd rather start with.